# Homework 3
# *Computer System Organization*
# ECE 361
# Fall 2026

## Overview

Homework 3 is about pointers: passing addresses so that a function can change the caller's variables or return more than one result, and walking arrays with pointer arithmetic. It is due on **Sun, Oct 18, 11:59pm**, pushed to `hw03/`. AI tools are allowed. Your Shine about your code is due by **Wed, Oct 21, 11:59pm**, without any supporting materials. Satisfactory / Not Yet, as before. Thursday's studio traces these pointers on paper first.

## Part 1: A pointer library

In `hw03/ptr.h` and `hw03/ptr.c`, write these four functions:

**Table 1. The four functions of the pointer library.**

| Function | What it does |
|---|---|
| `void swap_float(float *a, float *b)` | exchanges `*a` and `*b` |
| `int minmax(const float *a, int n, float *min, float *max)` | stores the smallest and largest of `a[0..n-1]` in `*min` and `*max` and returns 0; returns −1 and changes nothing if `n < 1` or any pointer is `NULL` |
| `const float *find_above(const float *a, int n, float threshold)` | returns a pointer to the first value greater than `threshold`, or `NULL` if there is none |
| `void reverse(float *a, int n)` | reverses `a[0..n-1]` in place; does nothing if `a` is `NULL` or `n < 2` |

- **Pointer arithmetic only:** inside `ptr.c`, walk the arrays with pointers (`p++`, `p < a + n`, `*p`), not with `a[i]`. `reverse` uses two pointers that move toward each other and calls `swap_float`.
- **const where the function only reads,** as in the prototypes above.
- **A comment per function in ptr.h:** what it takes, returns, and assumes.

## Part 2: Tests and a Makefile

- `hw03/tests/test_ptr.c` tests every function, at least: `swap_float`; `minmax` on several values, on one value, with `n = 0`, and with a `NULL` pointer (the outputs must stay unchanged); `find_above` with two matches (it returns the first), with none, and with a value equal to the threshold; `reverse` on an even and an odd length. It prints PASS or FAIL per test and a summary, and exits nonzero if any test fails.
- `hw03/Makefile`: `make` builds `ptr.o` and `tests/test_ptr` with `gcc -std=c11 -Wall -Wextra` and no warnings, `make test` runs the tests, `make clean` removes what was built. Each `.o` lists the headers it includes, as in HW2.

## Your project: a next step (not graded)

Scaffolding for your device, not part of HW3: it is not graded, and the HW3 Shine does not ask about it. It is the natural next step after Part 1, and it keeps your project moving. In `project/sensor.h` and `sensor.c`, write

- `int sensor_read(uint32_t *base, float *temp_c, float *rh_pct)`: reads `TEMP_REG` and `HUMID_REG` of iom361, converts them with the formulas on your project sheet, rounded to 0.1, and stores them in `*temp_c` and `*rh_pct`. Returns 0, or the iom361 return code if a read fails; then it changes nothing.

- `project/main.c`: after `iom361_initialize`, set the sensor to a value typical for your device with `_iom361_set_sensor1`, call `sensor_read`, and print the device name and the reading. `make` builds it with no warnings. (If you round with `roundf`, link with `-lm`.)

## Deliverable

Push by Sun, Oct 18, 11:59pm:

- `hw03/`: `ptr.h`, `ptr.c`, `tests/test_ptr.c`, `Makefile`, `README.md` (how to build and test), `AI_USAGE.md` (tools, for what, one thing a model got wrong)

## Shine

The window opens Mon, Oct 19, in the morning; due Wed, Oct 21, 11:59pm; about 10 to 15 minutes, about your own code. Expect questions like: “Describe, in words, the boxes and arrows for your `swap_float` when it is called. What would change without the `*`?” and “Why does your `minmax` return −1 instead of storing something, and why is a a `const float *`?” No notes, no second screen, no AI, no other person; webcam on.

## Requirements for Satisfactory

- Every deliverable is in your repository at the deadline.
- In `hw03/`: `make` builds with no errors and no warnings, `make test` runs and all tests pass, `make clean` works; `ptr.c` uses pointer arithmetic, not `a[i]`.
- Your Shine is completed by the deadline and meets the bar.